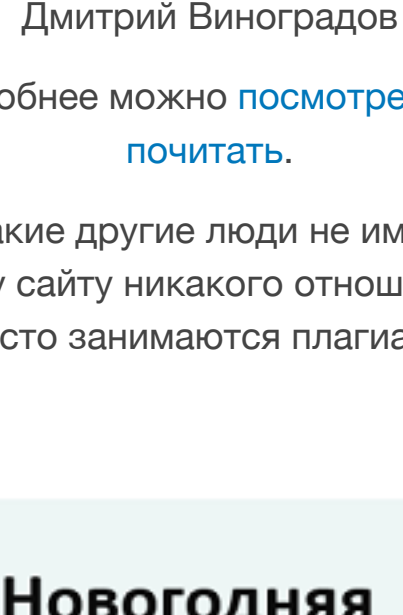


Language



Автор сайта



Дмитрий Виноградов

Подробнее можно [посмотреть](#) или [почитать](#).

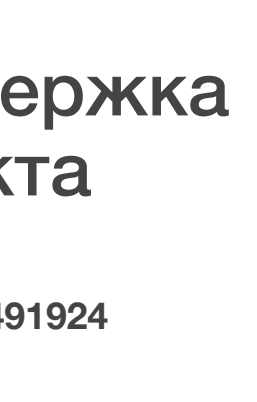
Никакие другие люди не имеют к этому сайту никакого отношения и просто занимаются плагиатом.

Новогодняя

скидка 50%
на набор курсов:

Jetpack Compose
Kotlin Coroutines
Dagger + Hilt
Testing

Социальные сети



В канале я публикую ссылки на интересные и полезные статьи по Android

В чате можно обсудить вопросы и проблемы, возникающие при разработке

Группа ВКонтакте



15 086 подписчиков



Подписаться

Поддержка проекта

Яндекс

410011180491924

WebMoney

R248743991365

Z551306702056

Paypal

Урок 8. Recomposition

14 декабря 2022

В этом уроке разбираемся, как `Recomposition` может оптимизировать перезапуски `Composable` функций.

`Recomposition` - еще один важный механизм в `Compose`. И мы его уже частично обсудили в уроке про `State`. Давайте снова проговорим основную мысль того урока.

Если `Composable` функция читает данные из `State`, то она автоматически подписывается на его изменения. Когда эти изменения происходят, функция перезапускает сама себя, чтобы считать из `State` новые значения и отобразить их на экране.

`Recomposition` - это и есть перезапуск `Composable` функции. Но у него есть еще один важный момент, который необходимо обсудить. Когда `Composable` функция перезапускается, она снова выполняет код внутри себя. А это значит, что она снова запускает все `Composable` функции, которые вызываются внутри нее. А эти функции вызывают свои вложенные функции и так далее. В итоге при перезапуске корневой `Composable` функции, мы получим перезапуск всего дерева функций. Кажется, это не очень правильно с точки зрения производительности. Но все не так плохо. Механизм `Recomposition` автоматически определяет, есть ли необходимость перезапускать `Composable` функцию. Ведь если ее входные данные не менялись, то ее перезапуск не имеет смысла. Она покажет на экране то же самое, что и раньше.

Давайте на примерах рассмотрим, как работает `Recomposition`.

Снова используем счетчик нажатий:

```
@Composable
fun HomeScreen(
    counter: State<Int>,
    onCounterClick: () -> Unit
) {
    val counterValue = counter.value
    Text(
        text = "Clicks: $counterValue",
        modifier = Modifier.clickable(onClick = onCounterClick)
    )
}
```

Мы получаем `State`, читаем из него значение и передаем в `Text`. А по клику на `Text` выполняем функцию `onCounterClick`, чтобы уведомить кого-то там снаружи, что был клик и пора менять значение `State`.

Тут в целом все хорошо и правильно. Максимально независимая `Composable` функция. Снаружи получает данные и обратно шлет клики. Но есть один момент, который можно улучшить. Все таки `HomeScreen` - это функция, которая отображает экран. Кроме счетчика нажатий, мы возможно захотим на этом экране отображать что-то еще. И тогда в `HomeScreen` будет куча кода, в котором будет трудно ориентироваться.

Поэтому цельные UI компоненты принято выносить в отдельные `Composable` функции. Счетчик нажатий - отличный кандидат, вынесем его в отдельную функцию. Можно в том же файле, где и `HomeScreen`.

Назовем эту новую функцию `ClickCounter`:

```
@Composable
fun ClickCounter(
    counter: State<Int>,
    onCounterClick: () -> Unit
) {
    val counterValue = counter.value
    Text(
        text = "Clicks: $counterValue",
        modifier = Modifier.clickable(onClick = onCounterClick)
    )
}
```

Код тот же, что и был.

Теперь вызовем `ClickCounter` в `HomeScreen`:

```
@Composable
fun HomeScreen(
    counter: State<Int>,
    onCounterClick: () -> Unit
) {
    ClickCounter(counter = counter, onCounterClick = onCounterClick)
}
```

`HomeScreen` передает в `ClickCounter` все, что нужно счетчику кликов: `State` и лямбду.

Обратите внимание, где теперь читается значение `State`:

```
val counterValue = counter.value
```

Это происходит уже не в `HomeScreen`, а в `ClickCounter`. А `HomeScreen` просто передает `State` в `ClickCounter`, но не читает его значение. Это значит, что при изменении значения `State` перезапускаться будет только функция `ClickCounter`. Давайте добавим логирование, чтобы убедиться в этом.

Логнуем вызов `HomeScreen`:

```
@Composable
fun HomeScreen(
    counter: State<Int>,
    onCounterClick: () -> Unit
) {
    Log.d(TAG, "HomeScreen")
    ClickCounter(counter = counter, onCounterClick = onCounterClick)
}
```

И логируем вызов `ClickCounter`:

```
@Composable
fun ClickCounter(
    counter: State<Int>,
    onCounterClick: () -> Unit
) {
    val counterValue = counter.value
    Log.d(TAG, "ClickCounter $counterValue")
    Text(
        text = "Clicks: $counterValue",
        modifier = Modifier.clickable {
            Log.d(TAG, "--- click ---")
            onCounterClick()
        }
    )
}
```

Задюю в логе будем выводить значение счетчика. А также логируем нажатия на текст.

Запускаем приложение и смотрим логи.

Сразу после запуска видим, что обе функции были вызваны:

```
HomeScreen
ClickCounter 0
```

Это логично. Мы в `Activity.onCreate` вызвали `HomeScreen`, а он уже вызвал `ClickCounter`

Теперь жмем несколько раз на текст:

```
--- click ---
ClickCounter 1
--- click ---
ClickCounter 2
--- click ---
ClickCounter 3
```

Перевызывается только функция `ClickCounter`. Потому что она читает значение из `State`, а значит подписывается на этот `State`, чтобы перезапуститься при изменении его значения.

А функция `HomeScreen` просто передает `State` в `ClickCounter`, но не читает значение этого `State`. А значит, не подписывается на него и не перезапускается при изменении его значения.

Таким образом `Recomposition` будет перезапускать только те функции, которые читают `State`. А функции, которые просто передают `State` дальше - не перезапускаются.

Входные данные

В начале урока я сказал, что механизм `Recomposition` автоматически определяет, есть ли необходимость перезапускать `Composable` функцию. Если входные данные функции не менялись с прошлого вызова, то не имеет смысла ее перезапускать. Она покажет на экране то же самое, что показала в прошлый раз. Чтобы увидеть, как это работает, немного поменяем и расширим наш пример.

Кроме счетчика кликов будем выводить на экран текст, который меняется в зависимости от количества сделанных кликов. Если кликов меньше 3, то показываем текст `More`. Если 3 или больше, то - `Enough`.

Предположим, что для показа таких текстов у нас уже есть своя отдельная `Composable` функция:

```
@Composable
fun InfoText(text: String) {
    Log.d(TAG, "InfoText $text")
    Text(text = text, fontSize = 24.sp)
}
```

Будем вызывать ее в `HomeScreen` вместе с `ClickCounter`:

```
@Composable
fun HomeScreen(
    counter: State<Int>,
    onCounterClick: () -> Unit
) {
    val counterValue = counter.value
    Log.d(TAG, "HomeScreen")
    Column {
        ClickCounter(counterValue = counterValue, onCounterClick = onCounterClick)
        InfoText(text = if (counterValue < 3) "More" else "Enough")
    }
}
```

Обратите внимание, что мы теперь не передаем `State` счетчика в `ClickCounter`. Мы в `HomeScreen` читаем значение `State` и уже это значение передаем в `ClickCounter`. А также используем его для определения текста, который хотим передать в `InfoText`.

Функция `ClickCounter` теперь выглядит так:

```
@Composable
fun ClickCounter(
    counterValue: Int,
    onCounterClick: () -> Unit
) {
    Log.d(TAG, "ClickCounter $counterValue")
    Text(
        text = "Clicks: $counterValue",
        modifier = Modifier.clickable {
            Log.d(TAG, "--- click ---")
            onCounterClick()
        }
    )
}
```

Вместо `State<Int>` получает сразу `Int` и передает его в `Text`.

В итоге `State` читается только в `HomeScreen`. А значит, меняя значение `State`, мы будем получать перезапуск `HomeScreen`. Но в итоге перезапуск функция `HomeScreen` вызовет функции `ClickCounter` и `InfoText` внутри себя и передаст им новые значения. Т.е. кажется, что смена `State` приведет к перезапуску всех трех функций. Но есть один важный нюанс.

Давайте смотреть логи.

Запускаем приложение:

```
HomeScreen
ClickCounter 0
InfoText More
```

Все функции запускаются первый раз. Счетчик отображает 0. Текст отображает `More`.

Начинаем кликать:

```
--- click ---
HomeScreen
ClickCounter 1
--- click ---
HomeScreen
ClickCounter 2
--- click ---
HomeScreen
ClickCounter 3
InfoText Enough
--- click ---
HomeScreen
ClickCounter 4
--- click ---
HomeScreen
ClickCounter 5
```

Видно, что `HomeScreen` перезапускается каждый раз, когда клик меняет значение в `State`. Это норма.

Также видно, что `HomeScreen` при этом вызывает функцию `ClickCounter`, чтобы показать актуальное количество кликов. Но совершенно не видно, что `HomeScreen` каждый раз вызывает функцию `InfoText`, чтобы передавать туда текст `More/Enough`. Мы видим только один вызов этой функции. И можно заметить, что этот вызов случился именно тогда, когда поменялся текст (`More -> Enough`), который `HomeScreen` передает в `InfoText`. И это не совпадение, а особенность `Recomposition`. При вызове `Composable` функция системы проверяет, поменялись ли входные параметры по сравнению с прошлым вызовом. Если не менялись, то нет смысла тратить ресурсы и время на вызов этой функции, потому что она покажет на экране то же самое, что уже показывал.

В нашем случае при запуске приложения в `InfoText` пришел текст `More`. Это был первый запуск функции.

Далее мы сделали первые два клика. `HomeScreen` перезапускался, получал количество кликов из `State`, вызывал `InfoText` и передавал туда текст `More`. Но `Compose` понял, что данные в `InfoText` передают те же, что и раньше, и проигнорировал вызов.

На третьем клике `HomeScreen` снова вызвал `InfoText` и передал туда текст `Enough`. `Compose` определил, что новые входные данные (`Enough`) отличаются от старых (`More`) и выполнил вызов `InfoText`. В результате текст на экране обновился.

Все последующие клики функция `InfoText` продолжала получать один и тот же текст `Enough`, поэтому эти вызовы были проигнорированы.

А вот в `ClickCounter` с самого начала каждый раз приходило значения, которые отличались от прошлых. Поэтому `Compose` не игнорировал вызовы `ClickCounter`.

Возможно, возникает вопрос. А что если входные параметры - не простые типы `Int` и `String`, а сложные объекты. На этот случай у `Compose` тоже есть определенная логика. Об этом будет отдельный урок.

За два последних урока мы выяснили, что `Composable` функция перезапускается при изменении значения в `State`, который читается в этой функции. Если же никакой `State` в функции не читается, но идет вызов от внешней `Composable` функции, то проверяются входные параметры, которые приходят снаружи. Если параметры отличаются от прошлого вызова, то будет выполнен перезапуск.

`Compose` старается максимально оптимизировать запуск `Composable` функций. Это еще одна причина (кроме производительности) почему не надо в `Composable` функциях менять значение в БД и выполнять прочие операции, которые напрямую меняют какое-то состояние снаружи. Потому что вы не можете заранее знать, сколько раз будет вызван эта функция. Даже если вы точно посчитали, что ваша `Composable` функция будет вызвана определенное количество раз за весь жизненный цикл экрана, то какой-то другой разработчик может добавить туда новых входных параметров или `State`. И это сломает ваши расчеты. Поэтому нам ни в коем случае нельзя связывать логику приложения на вызовы `Composable` функции.

Повторюсь, `Composable` функция принимает снаружи `State` или данные и отображает их на экране, а обратную связь осуществляет через лямбды. Сама функция напрямую не должна делать ничего важного.

Присоединяйтесь к нам в **Telegram**:

- в канале **StartAndroid** публикуются ссылки на новые статьи с сайта [startandroid.ru](#) и интересные материалы с хабра, [medium.com](#) и т.п.

- в чатах решаем возникающие вопросы и проблемы по различным темам: [Android](#), [Compose](#), [Kotlin](#), [RxJava](#), [Dagger](#), [Тестирование](#), [Performance](#)

- ну и если просто хочется поговорить с коллегами по разработке, то есть чат [Фудильный](#)

Назад

Вперед

Комментарии

RE: Урок 8. Recomposition — Игорь 27.01.2023 21:20
Небольшая опечатка в слове "HomeScreen" в тексте, после того как счетчик нажатий вынесли его в отдельную функцию.

Ответить

RE: Урок 8. Recomposition — Dmitry Vinogradov 03.02.2023 18:58
Спасибо, пофиксил!

Ответить

RE: Урок 8. Recomposition — Denis 06.02.2023 13:48
В предпоследнем предложении опечатка - принимает

Ответить

RE: Урок 8. Recomposition — Лют 15.02.2023 16:38
Не большая ридерка к словам: "А что если входные параметры - не примитивы, а объекты." В Kotlin примитивов нет.

Ответить

RE: Урок 8. Recomposition — Алескнадр 17.10.2023 13:30
В Kotlin примитивов нет, но при компиляции в байт код если это возможно будет компилировать в примитив

Ответить

RE: Урок 8. Recomposition — Иван 16.02.2023 14:03
Правильно понимаю? В принципе нет разницы, что передавать во "внутреннюю" функцию, `State` или же готовый ответ?
В обоих случаях функция перезапустится только если входные данные обновятся. Или `State` уже готовый "готовые" данные отличаются от имеющихся.
Благодарю за Ваш труд. Реально полезные курсы.

Ответить

RE: Урок 8. Recomposition — Dmitry Vinogradov 21.02.2023 22:27
Если передавать значение, и оно будет другим, чем в прошлый раз, то функция перевызовется.
Если передавать `State`, и в нем менять значение, то функция перевызовется только, если она читает значение из этого `State`.

Ответить

RE: Урок 8. Recomposition — Pavlo 14.01.2024 21:41
Оптимизация рекомпозиции работает только с простыми функциями типа `Text` или со всеми?
Когда из `ViewModel` получится `Flow` с объектом типа `(Int, Int, List)` и преобразовывается в `State`(Значение с помощью `collectAsState`). Отображение каждого из полей реализовано отдельными функциями, которые вызываются все даже когда изменяется только одно значение, то есть для остальных функций параметры не меняются.

Ответить

RE: Урок 8. Recomposition — Pavlo 14.01.2024 22:22
На тестовом примере счетчика кликов работает, а в реальном проекте что-то делаю, видимо, не так как надо

Ответить

RE: Урок 8. Recomposition — Dmitry Vinogradov 29.01.2024 20:06
Оптимизация работает для всех. Если функция перевызовется, значит один из входящих аргументов либо изменился, либо тип этого аргумента таков, что `Compose` не может однозначно определить - изменился ли он. Об этом еще будет отдельный урок.

Ответить

Обновить список комментариев

RSS лента комментариев этой записи

Добавить комментарий

Тема

😊 😄 😂 😃 😅 😆 😇 😈 😉 😊 😋 😌 😍 😎 😏 😐 😑 😒 😓 😔 😕 😖 😗 😘 😙 😚 😛 😜 😝 😞 😟 😠 😡 😢 😣 😤 😥 😦 😧 😨 😩 😪 😫 😬 😭 😮 😯 😰 😱 😲 😳 😴 😵 😶 😷 😸 😹 😺 😻 😼 😽 😾 😿 😽 😿 😺 😻 😼 😽 😾 😿 😽 😿 😺 😻 😼 😽 😾 😿

Отправить

Спамить